

Platformy programistyczne

Film Library

Autorzy:

Mariia Bulai (282655)

Amelia Ronka (280115)

1 Wprowadzenie

Celem zadania było zaprojektowanie i implementacja aplikacji webowej w technologii **.NET**.

Wybrany temat - **biblioteka multimediiów** przechowująca dane o filmach z możliwością dodawania, edycji i usuwania wpisów oraz integracją z zewnętrznym serwisem internetowym (The Movie Database, TMDb).

Aplikacja **FilmLibraryPP** umożliwia użytkownikom prowadzenie prywatnej kolekcji filmów: oznaczanie obejrzanych i planowanych tytułów, dodawanie recenzji i ocen, filtrowanie, wyszukiwanie, statystyki graficzne oraz import metadanych z TMDb.

Repozytorium projektu jest utrzymywane w systemie kontroli wersji **Git**. To sprawozdanie opisuje implementację aplikacji. Osobnym dokumentem jest **dokumentacja techniczna kodu** wygenerowana narzędziem DocFX (rozdział 12).

2 Cel i zakres projektu

2.1 Cel

Stworzenie w pełni funkcjonalnej aplikacji webowej do zarządzania osobistą biblioteką filmów, z persystencją danych w bazie relacyjnej, autoryzacją użytkowników oraz komunikacją z zewnętrznym API. Aplikacja uruchamiana jest lokalnie (Visual Studio / `dotnet run`).

2.2 Zakres funkcjonalny

- rejestracja i logowanie użytkowników z podziałem na role **Admin** i **User**;
- CRUD filmów (tworzenie, odczyt, edycja, usuwanie) przez formularze MVC;
- filtrowanie i wyszukiwanie po tytule, reżyserze i gatunku;
- listy „Do obejrzenia” i „Obejrzone”;
- tagi w relacji many-to-many;
- statystyki z wykresami (Chart.js) i wskaźnikami KPI;
- import metadanych filmu z TMDb (wyszukiwanie + przewypełnienie formularza);
- eksport biblioteki do pliku CSV;
- REST API `/api/films` z dokumentacją Swagger;
- testy jednostkowe (xUnit);
- dokumentacja techniczna kodu generowana przez DocFX (rozdział 12).

3 Wykorzystane technologie

Tabela 1: Stack technologiczny projektu

Obszar	Technologia
Język programowania	C#
Platforma	.NET 8.0
Framework webowy	ASP.NET Core MVC (Razor Views)
Frontend	Bootstrap 5, Bootstrap Icons, Chart.js
ORM	Entity Framework Core 8
Baza danych	PostgreSQL 16
Autoryzacja	ASP.NET Core Identity
Zewnętrzne API	TMDB (JSON przez <code>HttpClient</code>)
REST / dokumentacja API	Swagger (Swashbuckle)
Testy jednostkowe	xUnit + EF Core InMemory
Dokumentacja kodu	DocFX (komentarze XML)
Kontrola wersji	Git

4 Realizacja wymagań zadania

Poniższa tabela mapuje wymagania z treści zadania na konkretne elementy implementacji.

Tabela 2: Mapowanie wymagań zadania na implementację

Wymaganie	Realizacja w projekcie
Aplikacja w języku C# Interfejs ASP.NET	Cały backend napisany w C# (.NET 8). ASP.NET Core MVC z widokami Razor i Bootstrap.
Persystencja danych	PostgreSQL; dane zachowane po restarcie aplikacji i serwera bazy.
Entity Framework (ORM) Zapis i odczyt do bazy Kolekcje obiektów	<code>ApplicationDbContext</code> , migracje EF Core. Kontrolery MVC i <code>FilmsApiController</code> . <code>ICollection<Tag></code> , LINQ (<code>Where</code> , <code>GroupBy</code>).
Ręczne wprowadzanie danych Graficzna prezentacja danych	Formularze Create/Edit z walidacją modelu. Tabele list, KPI, wykresy Chart.js (statystyki).
Logowanie i role	ASP.NET Identity: role Admin / User.
Komunikacja z zewn. API	<code>TmdbService</code> - HTTP do <code>api.themoviedb.org</code> .
Format JSON/HTML/XML	JSON (TMDB, REST API); HTML (widoki Razor).
Walidacja danych	Atrybuty <code>[Required]</code> , <code>[Range]</code> , jQuery Validation.
Obsługa wyjątków	<code>try/catch</code> w serwisie TMDB; <code>ExceptionHandler</code> .

Tabela 2: Mapowanie wymagań zadania na implementację (cd.)

Wymaganie	Realizacja w projekcie
Dokumentacja generatora	DocFX + komentarze XML (GenerateDocumentationFile).
Konteneryzacja	Dockerfile (multi-stage build .NET 8) oraz docker-compose.yml (PostgreSQL + aplikacja, healthcheck, wolumen na dane).
Dodatkowe: testy jednostkowe	27 testów xUnit (walidacja, mapper TMDB, M2M, seed).

5 Architektura aplikacji

Projekt ma strukturę typową dla ASP.NET Core MVC z warstwą usług i osobnym projektem testowym.

Listing 1: Struktura repozytorium

```
FilmLibraryPP.sln
|
|-- FilmLibraryPP/                                # aplikacja MVC
|   |-- Controllers/
|   |   '-- Api/                                    # FilmsApiController
|   |-- Models/
|   |   '-- Api/                                    # DTO dla REST API
|   |-- Data/                                       # ApplicationDbContext, seedy
|   |-- Services/Tmdb/                             # klient TMDB + mapper
|   |-- Migrations/                               # migracje EF Core
|   |-- Views/                                     # Razor: Home, Films,
|       Statistics, Shared
|   |-- wwwroot/                                   # Bootstrap, jQuery, CSS
|   |-- Program.cs
|   |-- Dockerfile
|   '-- FilmLibraryPP.csproj
|
|-- FilmLibraryPP.Tests/                           # testy xUnit
|-- docs/                                           # konfiguracja DocFX + strony
    MD
|-- docker-compose.yml
|-- .env.example
'-- README.md
```

5.1 Przepływ żądania

1. Żądanie HTTP trafia do kontrolera MVC lub API.
2. Middleware Identity weryfikuje uwierzytelnienie i autoryzację.
3. Kontroler korzysta z `ApplicationDbContext` (EF Core) lub `ITmdbService`.
4. Dane są mapowane na modele widoku / DTO i zwracane jako HTML lub JSON.

Przy starcie aplikacji (`Program.cs`) wykonywane są migracje bazy oraz seed ról i konta administratora.

6 Warstwa danych i Entity Framework

6.1 Model domeny

Główna encja `Film` zawiera m.in.: tytuł, rok, gatunek, reżyser, ocenę, opis, recenzję, datę obejrzenia, flagę `IsWatched`, URL plakatu oraz powiązanie z właścicielem (`OwnerId` → `ApplicationUser`).

Encja `Tag` jest powiązana z `Film` relacją **many-to-many** (przez tabelę pośrednią `FilmTags`).

6.2 Kontekst bazy danych

Klasa `ApplicationDbContext` dziedziczy po `IdentityDbContext<ApplicationUser>`, łącząc tabele domeny z tabelami ASP.NET Identity. Konfiguracja relacji odbywa się w metodzie `OnModelCreating`.

6.3 Persystencja

- silnik bazy: PostgreSQL 16 (Npgsql);
- migracje EF Core stosowane automatycznie przy starcie;
- dane seedowane przez `SeedData` i `IdentitySeed`;
- baza PostgreSQL działa lokalnie na maszynie deweloperskiej (port 5432).

7 Interfejs użytkownika

Interfejs oparty jest na widokach Razor z biblioteką Bootstrap 5. Główne ekrany:

- **Lista filmów** - karty z plakatami, filtrowaniem i wyszukiwaniem;
- **Szczegóły / edycja / usuwanie** - formularze z walidacją po stronie klienta i serwera;
- **Do obejrzenia / Obejrzane** - listy filtrowane po polu `IsWatched`;
- **Wyszukiwanie TMDb** - import metadanych z zewnętrznego serwisu;
- **Statystyki** - 4 wskaźniki KPI oraz 3 wykresy: gatunki, rozkład ocen, obejrzane wg roku.

Wykresy renderowane są przez bibliotekę **Chart.js** na podstawie danych przygotowanych w `StatisticsController` z użyciem LINQ (`GroupBy`, `Count`). Przykładowe widoki interfejsu przedstawiono w rozdziale [15](#).

8 Autoryzacja i role użytkowników

System wykorzystuje **ASP.NET Core Identity** z dwiema rolami:

- **User** - widzi i zarządza wyłącznie własną biblioteką filmów (rys. [5](#));

- **Admin** - ma dostęp do wszystkich filmów wszystkich użytkowników oraz widzi przy każdym wpisie adres e-mail właściciela (rys. 4).

Kontrolery MVC i API oznaczone są atrybutem `[Authorize]`. Hasła muszą spełniać politykę złożoności (min. 8 znaków, cyfra, wielka i mała litera). Konto administratora tworzone jest przy pierwszym uruchomieniu na podstawie konfiguracji `AdminUser:Password`.

9 Integracja z zewnętrznym API (TMDB)

Aplikacja komunikuje się z serwisem **The Movie Database** (TMDB) przez typowany klient HTTP (`HttpClient`) zarejestrowany w DI jako `ITmdbService`.

- wyszukiwanie filmów: endpoint `search/movie`;
- pobieranie szczegółów: endpoint `movie/{id}`;
- format wymiany danych: **JSON**;
- autoryzacja: token Bearer w nagłówku HTTP;
- mapowanie odpowiedzi JSON na model domeny: klasa `TmdbMapper`.

Serwis obsługuje błędy sieciowe i parsowania JSON, loguje je i zwraca bezpieczne wartości domyślne, dzięki czemu awaria TMDB nie powoduje crashu aplikacji.

Po wybraniu filmu z wyników wyszukiwania TMDB formularz dodawania zostaje automatycznie wypełniony metadanymi (tytuł, rok, reżyser, gatunek, ocena, opis, URL plakatu). Użytkownik może je jeszcze edytować przed zapisem do bazy (rys. 6). Przy ręcznym dodawaniu filmu (bez importu z TMDB) wszystkie pola formularza pozostają puste.

10 REST API własne

Oprócz interfejsu MVC aplikacja udostępnia REST API pod ścieżką `/api/films`:

- GET `/api/films` - lista z paginacją;
- GET `/api/films/{id}` - szczegóły filmu;
- POST `/api/films` - utworzenie;
- PUT `/api/films/{id}` - aktualizacja;
- DELETE `/api/films/{id}` - usunięcie.

Odpowiedzi zwracane są w formacie JSON. Dokumentacja interaktywna dostępna jest przez **Swagger UI** (`/swagger`). Dla ścieżek API stosowane są kody HTTP 401/403 zamiast przekierowań HTML na stronę logowania.

11 Walidacja danych i obsługa wyjątków

11.1 Walidacja

- atrybuty walidacji na modelu `Film` (`Required`, `StringLength`, `Range`, `Url`);
- walidacja po stronie klienta: jQuery Validation + unobtrusive;
- walidacja danych z TMDB: sprawdzanie kodów HTTP i poprawności deserializacji JSON;
- walidacja parametrów API (np. ograniczenie `pageSize` do 1–100).

11.2 Obsługa wyjątków

- w środowisku produkcyjnym: `UseExceptionHandler("/Home/Error");`
- w serwisie TMDb: przechwytywanie `HttpRequestException`, `TaskCanceledException`, `JsonException` z logowaniem przez `ILogger`;
- seed administratora: rzucanie `InvalidOperationException` przy nieudanym tworzeniu konta.

12 Dokumentacja techniczna kodu (DocFX)

W projekcie zastosowano **DocFX** - narzędzie Microsoftu do tworzenia statycznych stron HTML z komentarzy XML w C#.

12.1 Źródła dokumentacji

DocFX łączy dwa typy materiałów:

1. **Komentarze XML** (`/// <summary>...`) w plikach `.cs` - generowane do pliku `FilmLibraryPP.xml` dzięki opcji `<GenerateDocumentationFile>true</GenerateDocumentationFile>` w pliku projektu;
2. **Strony Markdown** w katalogu `docs/` - opis projektu i instrukcja uruchomienia (`index.md`, `docs/introduction.md`, `docs/getting-started.md`).

Przykład komentarza w kodzie (model `Film`):

```
/// <summary>
/// Podstawowa encja domeny - pojedynczy film w bibliotece
///   użytkownika.
/// </summary>
public class Film
{
    /// <summary>Tytuł filmu (wymagany, max 200 znaków).</summary>
    >
    [Required(ErrorMessage = "Tytuł jest wymagany.")]
    public string Title { get; set; } = string.Empty;
}
```

Udokumentowane zostały m.in.: `Film`, `Tag`, `ApplicationUser`, `ApplicationDbContext`, `IdentitySeed`, `FilmsApiController`, `ITmdbService`, `TmdbMapper` oraz DTO API.

12.2 Konfiguracja i generowanie

Plik `docs/docfx.json` wskazuje projekt `FilmLibraryPP.csproj` jako źródło metadanych oraz definiuje szablon wyjściowy (`default`, `modern`). Narzędzie DocFX instaluje się jednorazowo jako globalne narzędzie .NET:

```
dotnet tool install -g docfx
```

Generowanie dokumentacji z katalogu głównego repozytorium:

```
docfx docs/docfx.json
```

Wynik: statyczna strona HTML w katalogu `docs/_site/`. Podgląd w przeglądarce:

```
docfx docs/docfx.json --serve
```

Serwer podglądu uruchamia się pod adresem <http://localhost:8080>.

12.3 Zawartość wygenerowanej strony

Wygenerowana dokumentacja (rys. 1) zawiera:

Tabela 3: Struktura dokumentacji DocFX

Sekcja	Opis
Docs / Wprowadzenie	krótki opis projektu i celu aplikacji
Docs / Pierwsze uruchomienie	instrukcja <code>dotnet run</code> , User Secrets, testy
API	referencja typów: klasy, metody, właściwości, parametry
Wyszukiwarka	przeszukiwanie całej dokumentacji (Search)

Na stronie głównej widoczny jest m.in. stack technologiczny (ASP.NET Core MVC 8, EF Core, PostgreSQL, Identity, TMDB, xUnit, Swagger). Zakładka **API** zawiera pełną referencję namespace `FilmLibraryPP` z opisami przeniesionymi z komentarzy XML.

Ten sam plik XML dokumentacji wykorzystuje też **Swagger** — opisy endpointów `/api/films` w interfejsie `/swagger` pochodzą z komentarzy nad metodami w `FilmsApiController`.



Rysunek 1: Strona główna dokumentacji DocFX wygenerowanej z komentarzy XML i plików Markdown (`docfx -serve`, `localhost:8080`).

13 Uruchomienie aplikacji

Aplikacja udostępnia dwa równoważne sposoby uruchomienia — lokalnie przez `dotnet run` oraz w kontenerze przez `docker compose up`. W repozytorium znajdują się komplety plików konfiguracyjnych dla obu wariantów (`Dockerfile`, `docker-compose.yml`, `.env.example`, `appsettings.json`, User Secrets).

13.1 Wymagania

- .NET 8 SDK;
- PostgreSQL 16+ (lokalna instalacja);
- edytor: Visual Studio, Visual Studio Code lub inny z obsługą C#.

13.2 Konfiguracja

Hasła i klucze API przechowywane są w **User Secrets** (katalog FilmLibraryPP/):

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection" ^
    "Host=localhost;Port=5432;Database=filmlibrary;Username=
    postgres;Password=HASLO"

dotnet user-secrets set "AdminUser:Password" "Admin123!"
dotnet user-secrets set "Tmdb:BearerToken" "TOKEN_TMDB"
```

13.3 Start aplikacji

```
dotnet restore
dotnet run --project FilmLibraryPP
```

Przy pierwszym uruchomieniu EF Core automatycznie stosuje migracje i seeduje dane. Aplikacja dostępna jest pod adresem <https://localhost:7073> lub <http://localhost:5269>.

13.4 Wariant Docker

Alternatywnie aplikację można uruchomić w kontenerach. Wymaga to zainstalowanego **Docker Desktop**. Konfiguracja sekretów odbywa się przez plik `.env` (szablon: `.env.example`):

```
Copy-Item .env.example .env          # PowerShell
docker compose up --build
```

Docker Compose uruchamia dwa serwisy: `db` (PostgreSQL 16 Alpine z healthcheckiem) oraz `web` (aplikacja ASP.NET Core). Aplikacja dostępna jest pod adresem <http://localhost:8080>. Dane bazy przechowywane są w nazwanym wolumenie `postgres_data` i przeżywają restarty kontenerów.

14 Testy jednostkowe

Projekt testowy `FilmLibraryPP.Tests` wykorzystuje framework **xUnit** oraz bazę InMemory EF Core do testów integracyjnych warstwy danych.

Tabela 4: Przegląd testów jednostkowych

Plik testowy	Zakres
FilmValidationTests.cs	walidacja modelu Film
TmdbMapperTests.cs	mapowanie odpowiedzi TMDB
TagsM2MTests.cs	relacja many-to-many Film-Tag
SeedDataTests.cs	poprawność danych seed
Łącznie	27 testów, wszystkie przechodzą

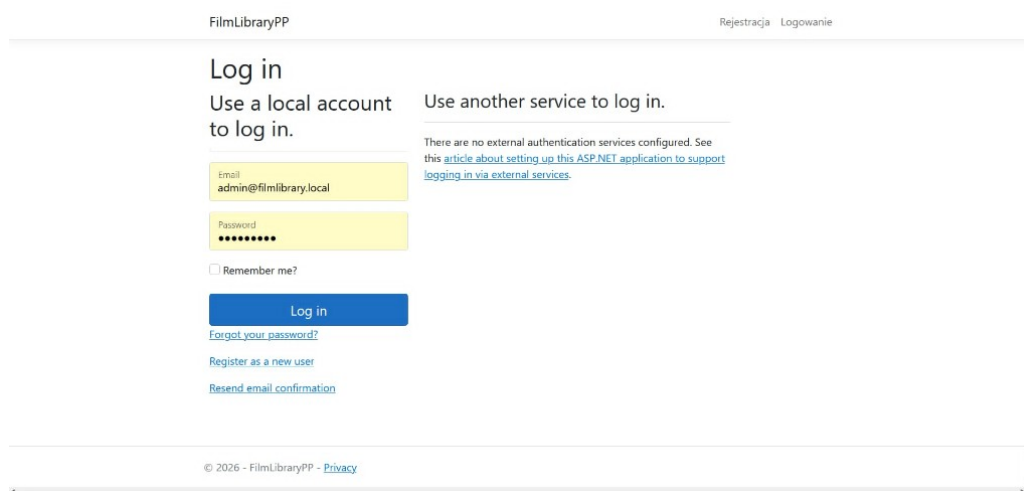
Uruchomienie testów:

```
dotnet test
```

15 Interfejs aplikacji

Poniżej przedstawiono wybrane ekrany aplikacji uruchomionej lokalnie. Ilustrują one realizację wymagań dotyczących interfejsu użytkownika, autoryzacji, integracji z TMDB oraz prezentacji danych w formie wykresów.

15.1 Logowanie i rejestracja

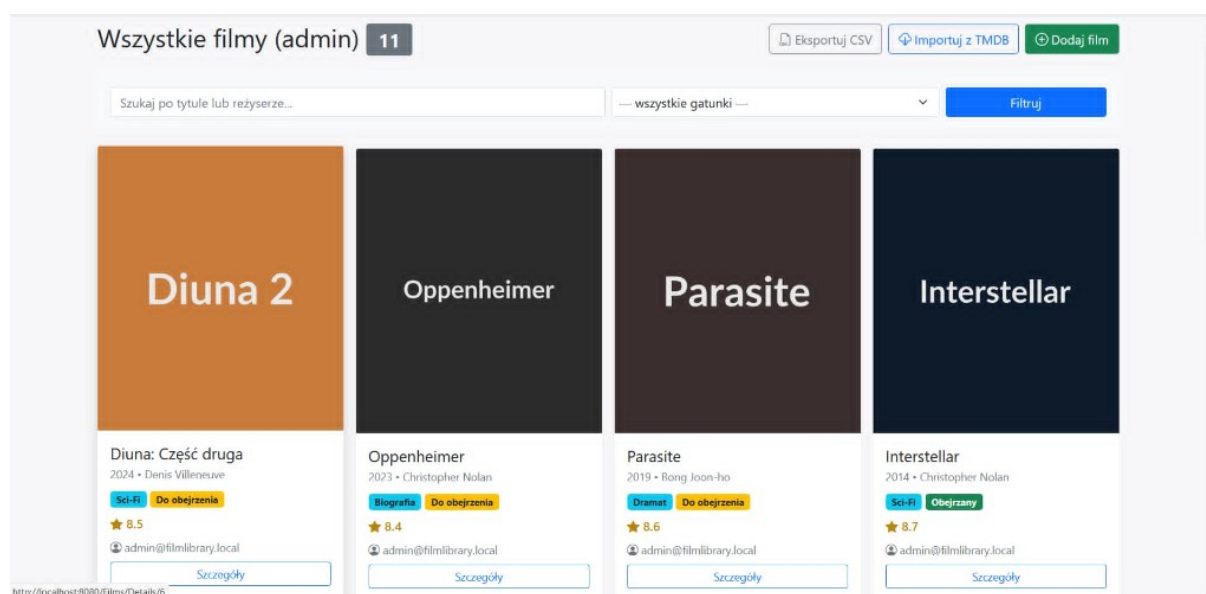


Rysunek 2: Strona logowania (ASP.NET Core Identity). Użytkownik loguje się adresem e-mail i hasłem.

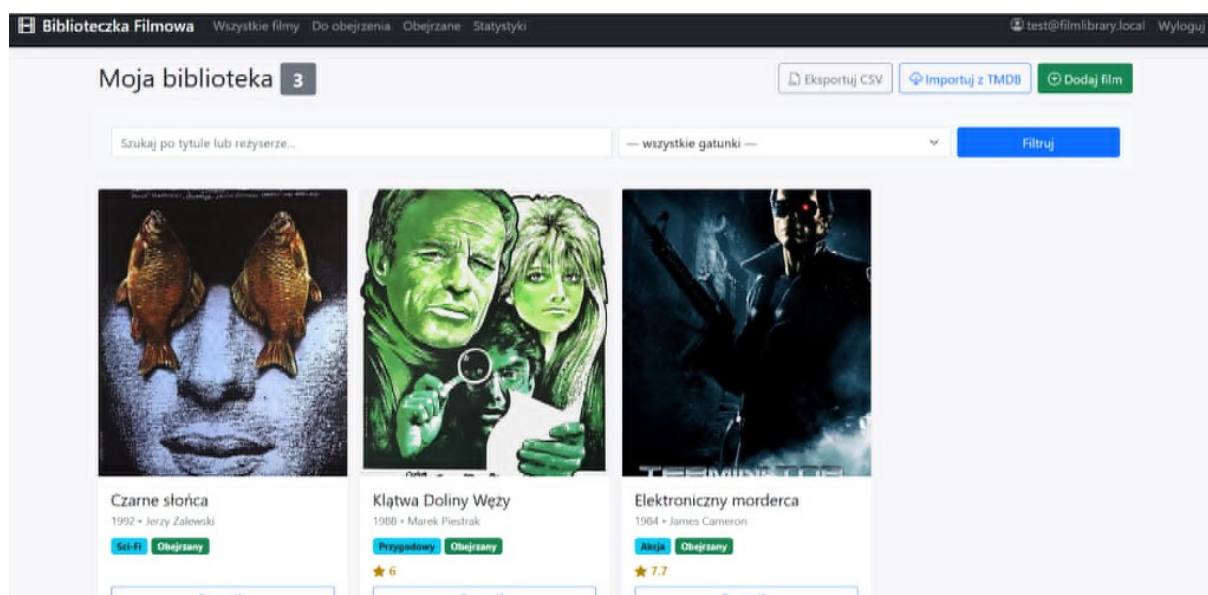
Rysunek 3: Strona rejestracji nowego konta użytkownika.

15.2 Widok administratora i użytkownika

Administrator widzi **wszystkie filmy** w systemie - zarówno własne, jak i należące do innych kont. Przy każdej karcie filmu wyświetlany jest adres e-mail właściciela. Zwykły użytkownik widzi wyłącznie **własną bibliotekę**.



Rysunek 4: Widok administratora: lista wszystkich filmów z podpisem właściciela (admin@filmlibrary.local itd.), filtrowanie i import z TMDb.



Rysunek 5: Widok użytkownika (test@filmlibrary.local): prywatna biblioteka z trzema filmami, wyszukiwanie i filtrowanie po gatunku.

15.3 Dodawanie filmu

(+) Dodaj nowy film

⚙️ Załadowano dane z TMDb dla „Wielka heca Bowfingera”. Sprawdź pola, w razie potrzeby popraw, i kliknij „Zapisz film”.

Tytuł Wielka heca Bowfingera		Rok 1999
Reżyser Frank Oz	Gatunek Komedia	Ocena 6.2

Opis
Właścicielem podupadającej wytwórni filmowej Bobby Bowfinger trafia na znakomity scenariusz filmu. Niestety, gwiazdor kina akcji, Kit Ramsey, odmawia mu udziału w tej produkcji. Zarozumiالى i egoistyczny Kit cierpi na depresję. Ale przebiegły Bobby nie poddaje się i postanawia nakręcić film z Kitem bez jego wiedzy. Pomysłowość reżysera nie ma granic, to prowadzi do przezabawnych gagów i sytuacji.

Recenzja

URL plakatu
https://image.tmbd.org/t/p/w500/8s6ve8F3mli29RNCIqWnB3UAPtg.jpg

Tagi (rozdzielone przecinkami)
np. Ulubione, Klasyka

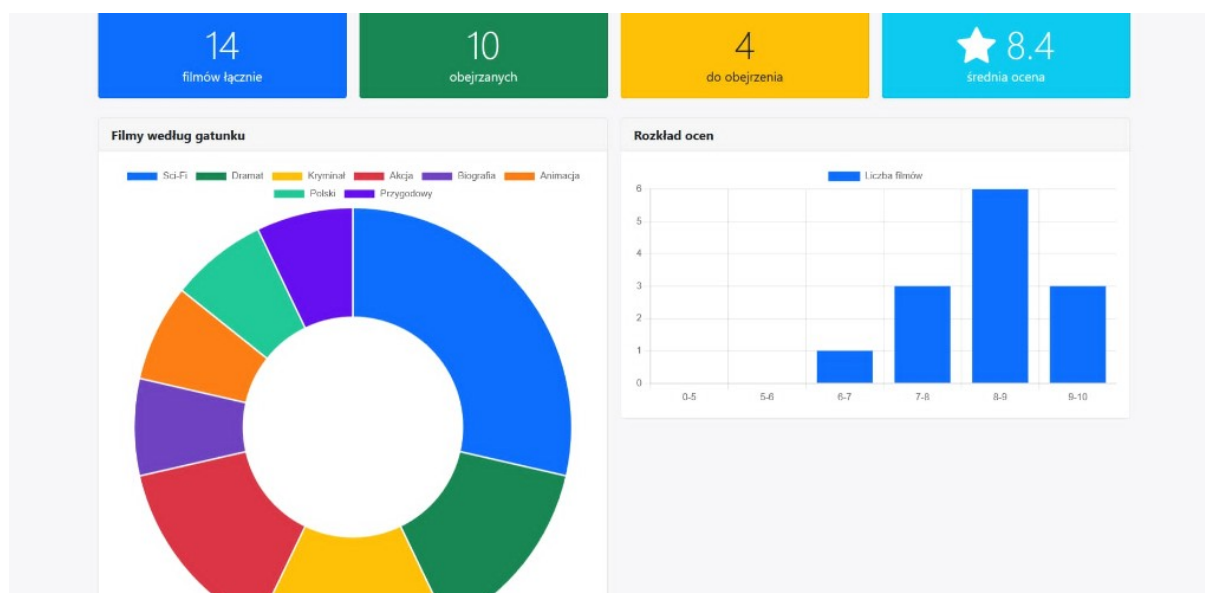
☐ Obejrzone

Data obejrzenia
dd . mm . rrrr

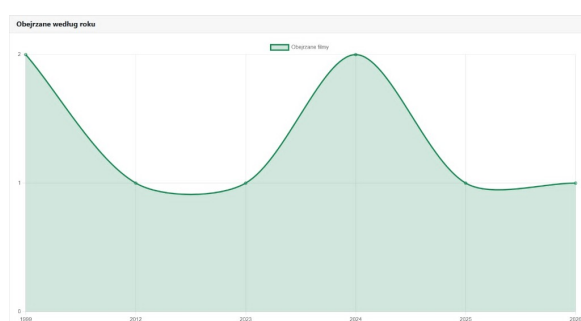
Rysunek 6: Formularz dodawania filmu po imporcie z TMDb - pola wypełnione automatycznie (np. „Wielka heca Bowfingera”), ale nadal edytowalne przed zapisem. Przy ręcznym dodawaniu formularz startuje z pustymi polami.

15.4 Statystyki i wykresy

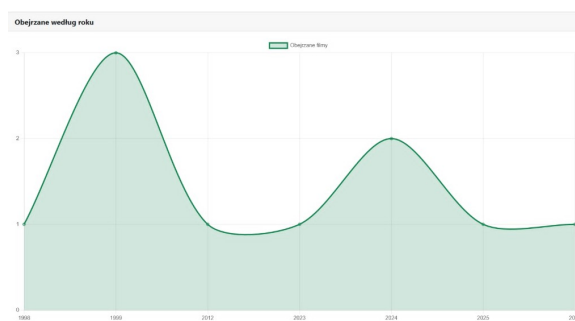
Strona statystyk prezentuje cztery wskaźniki KPI (łączna liczba filmów, obejrzone, do obejrzenia, średnia ocena) oraz wykresy Chart.js: rozkład gatunków (donut), rozkład ocen (słupki) i obejrzone filmy według roku (linia).



Rysunek 7: Statystyki użytkownika: KPI oraz wykresy gatunków i rozkładu ocen.



(a) Widok użytkownika - obejrzone według roku



(b) Widok administratora - agregacja wszystkich użytkowników

Rysunek 8: Wykres liniowy „Obejrzone według roku porównanie zakresu danych użytkownika i administratora”

16 Podsumowanie

Zrealizowano aplikację webową **Biblioteka Filmowa** spełniającą wymagania zadania z przedmiotu *Platformy programistyczne*. Aplikacja działa lokalnie (`dotnet run` + PostgreSQL); integracja z TMDb realizuje wymóg komunikacji sieciowej z zewnętrznym API.

Kluczowe osiągnięcia projektu:

- pełna persystencja danych w PostgreSQL przez Entity Framework Core;
- interfejs MVC z formularzami, tabelami i wykresami;
- autoryzacja z podziałem na role Admin/User;
- komunikacja z TMDb (JSON) z walidacją i obsługą błędów;
- własne REST API z dokumentacją Swagger;
- konteneryzacja przez Docker Compose (PostgreSQL + aplikacja);
- 27 testów jednostkowych xUnit;
- dokumentacja kodu wygenerowana przez DocFX z komentarzy XML.

Sprawozdanie (Overleaf) opisuje projekt. Dokumentacja techniczna kodu (DocFX, rys. 1) stanowi osobny artefakt w katalogu `docs/_site/` i może być wygenerowana ponownie poleceniem `docfx docs/docfx.json`.